BCSE497J PROJECT-I · FINAL REVIEW

# Context-Aware Personal AI Assistant

Plug in any model, score it on public tasks, route every question by that score, and keep memory and files on your own computer.

**Nischay N · Tejas K M · Tenidhar Reddy Chintham**

Guide: Prof. Mary Mekala A · School of Computer Science and Engineering, VIT Vellore

November 2026

# What Review 2 asked us to fix

*“The diagrams look AI-generated and the actor graph is off.”*

Every diagram is generated by a script from the code, in standard UML and Yourdon notation. Class diagrams are parsed from the source; use cases match real app controls.

*“What is the score name? How is each model graded?”*

The Routing Utility Score (RUS), built from a measured scorecard per model and task type: accuracy with a 95% interval, real cost, real latency.

*“Direct questions show 100%. How is it 100%?”*

It was measured on our own prompts. On 200 unseen public prompts the classifier now scores 97%, and every rate shows its n and 95% interval.

# Three goals, one assistant

OPEN

## Any model can be plugged in

Local (Ollama) or cloud (Anthropic, Gemini, any OpenAI-compatible API). One entry in models.yaml, one calibration run.

PRIVATE

## Your data stays on your computer

Memory, documents, history and scorecards live in a local folder. A question that carries any of them is answered by a local model only.

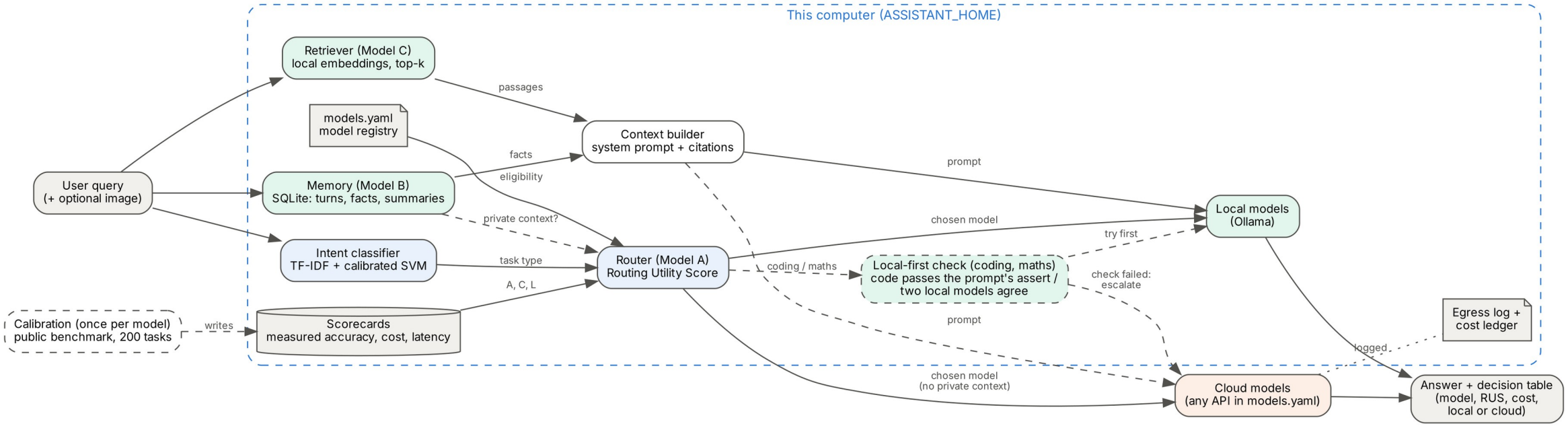
CHEAP

## Pay only for what a question needs

Each question goes to the model with the best measured trade-off of accuracy, cost and speed for its task type.

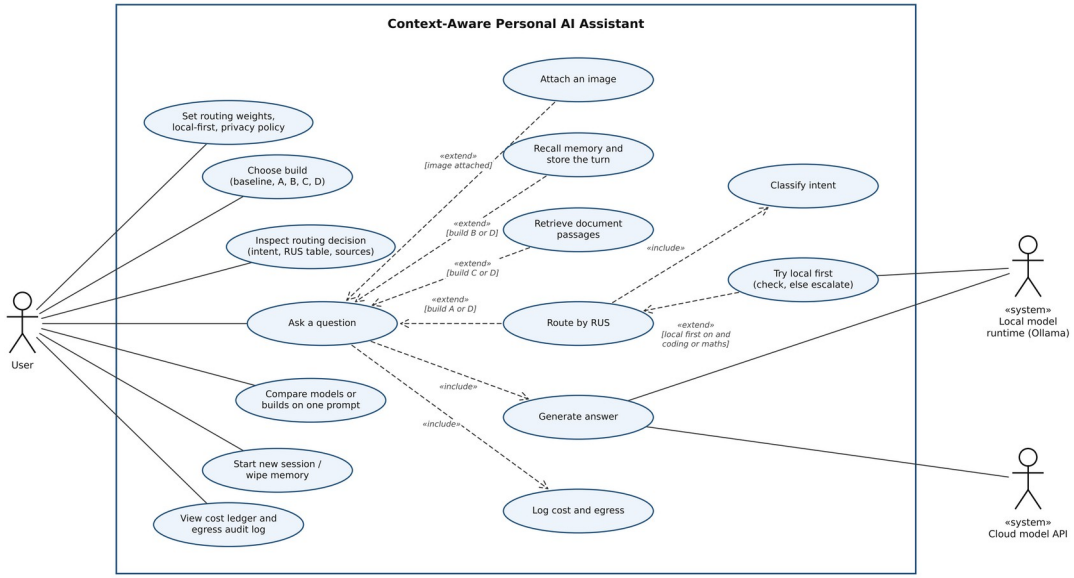
**95% cheaper** than always using Claude Sonnet 5.5, at the same accuracy: 89.5% vs 89% on 200 unseen public tasks.

# Architecture: one pipeline, three switches

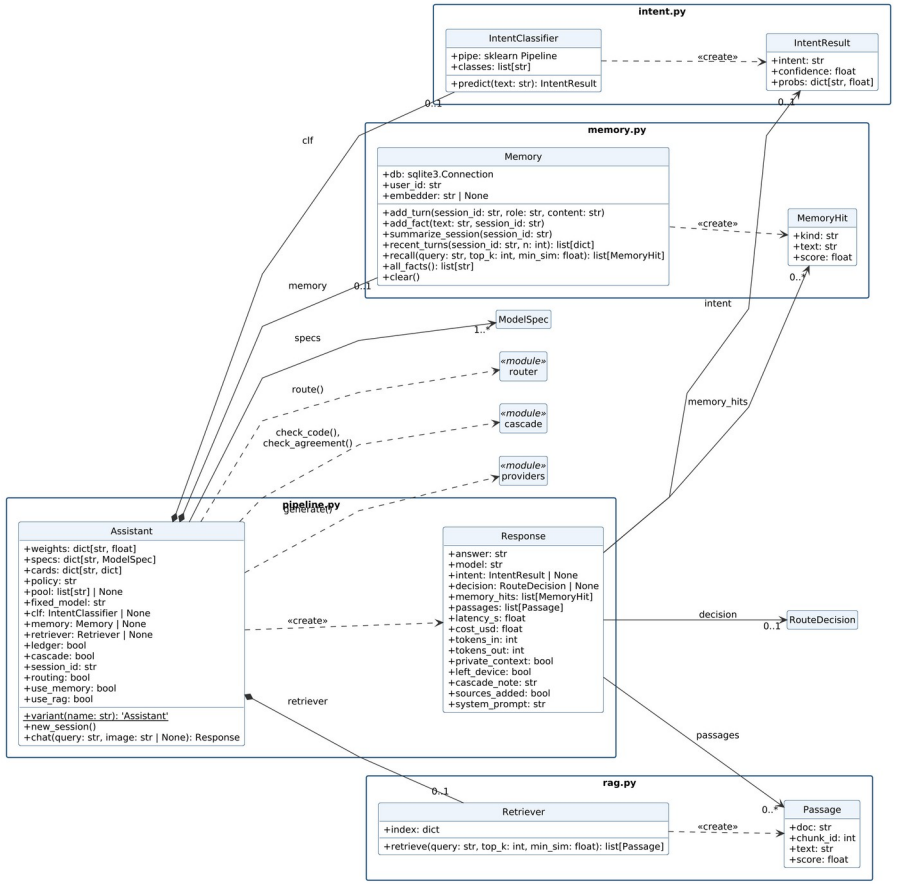



Routing, memory and RAG are switches on one Assistant class, so builds A to D are the same code. Everything inside the dashed box runs on the user's computer; only a prompt with no private context can reach a cloud model.

# Design in standard notation, generated from the code




Use cases (UML 2.5): every use case is a real app control; «extend» conditions name the build that enables it




Classes parsed from the source with types; composition, associations and dependencies as in the code

# Plugging in a model takes two steps

### 1 · Register it in models.yaml

```
- name: claude-haiku-5.5
  provider: anthropic
  model: claude-haiku-5-5
  api_key_env: ANTHROPIC_API_KEY
  local: false
  vision: true
  price_in: 0.10    # $ per 1M tokens
  price_out: 0.50
```

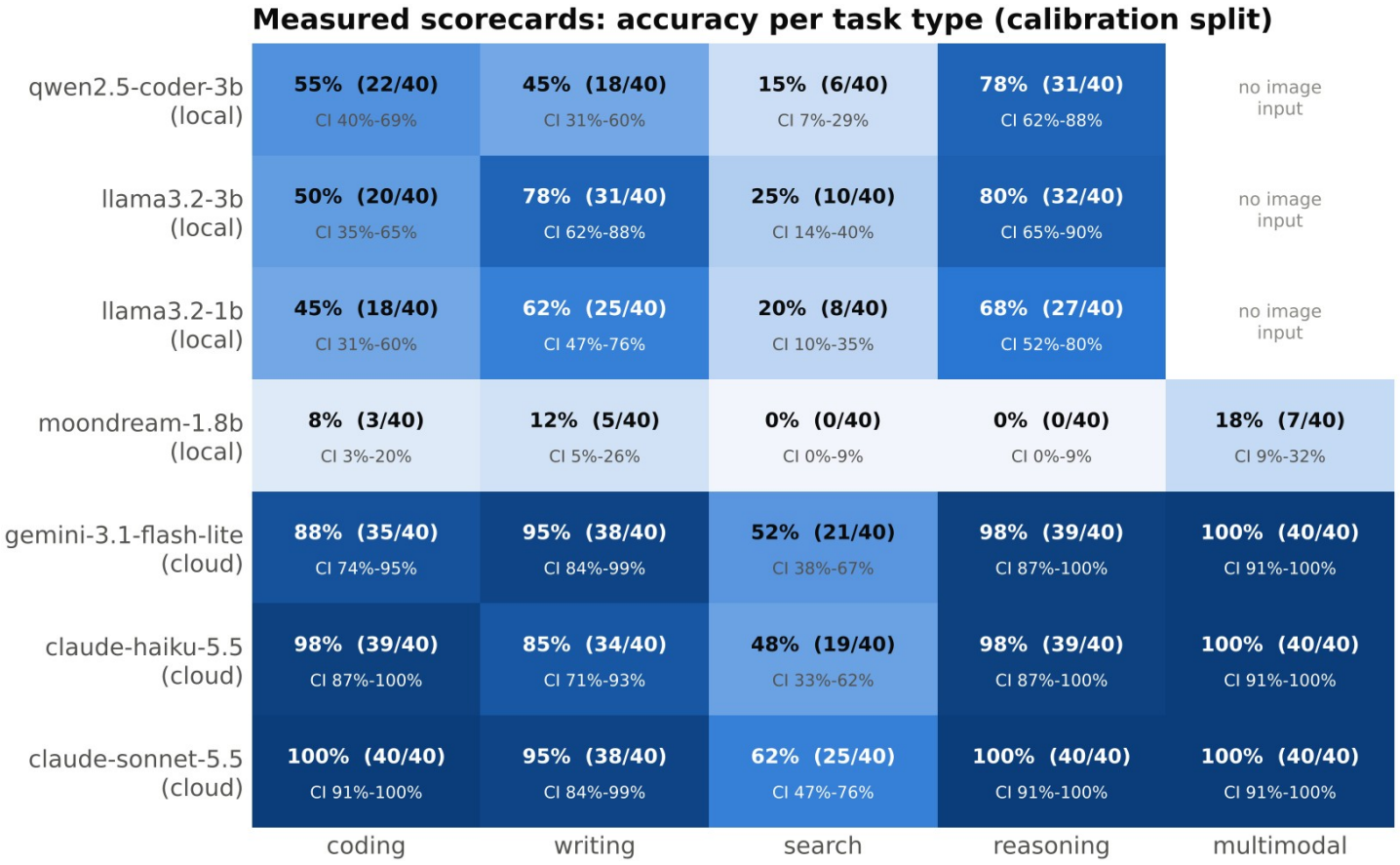
### 2 · Calibrate it

```
python -m assistant calibrate --model claude-haiku-
5.5
```

| provider | covers |
|---|---|
| ollama | any local model |
| openai | OpenAI, Groq, Together, LM Studio, vLLM |
| anthropic | Claude models |
| gemini | Google Gemini |

Calibration runs the 200 public calibration tasks and writes the model's scorecard. The router uses it on the next question. No code changes.

# A measured scorecard per model and task type




n = 40 tasks per cell (MBPP, IFEval, NQ-open, GSM8K, generated images). 95% Wilson intervals. Source: results/final/eval.json

## What a scorecard holds

- tasks tried and correct (40 per cell)
- accuracy with its 95% Wilson interval
- median latency, measured
- mean cost per question: real tokens × list price

Cloud models lead on every type. The best local model reaches 80% on maths but 25% on factual questions.

# The score: Routing Utility Score (RUS)

$$RUS = w_a \cdot A + w_c \cdot (1 - C / C_{max}) + w_l \cdot (1 - L / L_{max})$$

### A · accuracy

Lower end of the 95% Wilson interval. 40/40 gives 0.91; 4/4 only 0.51, so small samples are not over-trusted.

### C · cost

Mean measured cost per question. Budget C_max = $0.002; at or above it the term is 0. Local models cost $0.

### L · latency

Median seconds per answer on our machine. Budget L_max = 30 s.

Balanced weights 0.6 / 0.2 / 0.2, user-adjustable · highest RUS wins, ties go to the cheaper model · private context → local models only

# Worked example: a coding question, balanced weights

| Model | Correct / 40 | A (Wilson low) | Cost / question | Latency | 0.6A + 0.2C′ + 0.2L′ | RUS |
|---|---|---|---|---|---|---|
| **Claude Haiku 5.5** | 39 | 0.871 | $0.00013 | 2.2 s | 0.523 + 0.187 + 0.185 | 0.895 |
| Gemini 3.1 Flash-Lite | 35 | 0.739 | $0.00012 | 1.3 s | 0.443 + 0.188 + 0.191 | 0.822 |
| Claude Sonnet 5.5 | 40 | 0.912 | $0.00162 | 2.1 s | 0.547 + 0.038 + 0.186 | 0.772 |
| qwen2.5-coder-3b (local) | 22 | 0.398 | $0 | 5.0 s | 0.239 + 0.200 + 0.167 | 0.606 |

Claude Haiku 5.5 wins. Sonnet is 1 task more accurate, but it spends 81% of the cost budget, so its cost term nearly vanishes. Move the slider to quality-first and Sonnet wins instead.

C′ = 1 − C / $0.002 · L′ = 1 − L / 30 s · coding calibration scorecards

# 400 public tasks, graded by rules, not by AI

| Task type | Source | Marked correct when |
|---|---|---|
| Coding | MBPP | code passes the dataset's own unit tests |
| Maths | GSM8K | final number equals the gold number |
| Factual search | Natural Questions (open) | the answer contains an accepted answer |
| Writing | IFEval | every stated instruction holds (length, format, keywords) |
| Images | 80 generated charts and shapes | matches the known answer |

## 200 calibration

40 per type · build the scorecards

## 200 test

40 per type · never used to build scorecards or train the classifier

# Result: Sonnet's accuracy at a fraction of the cost

**89.5%**

routed (balanced), 179/200
95% CI 84.5%–93%

**89%**

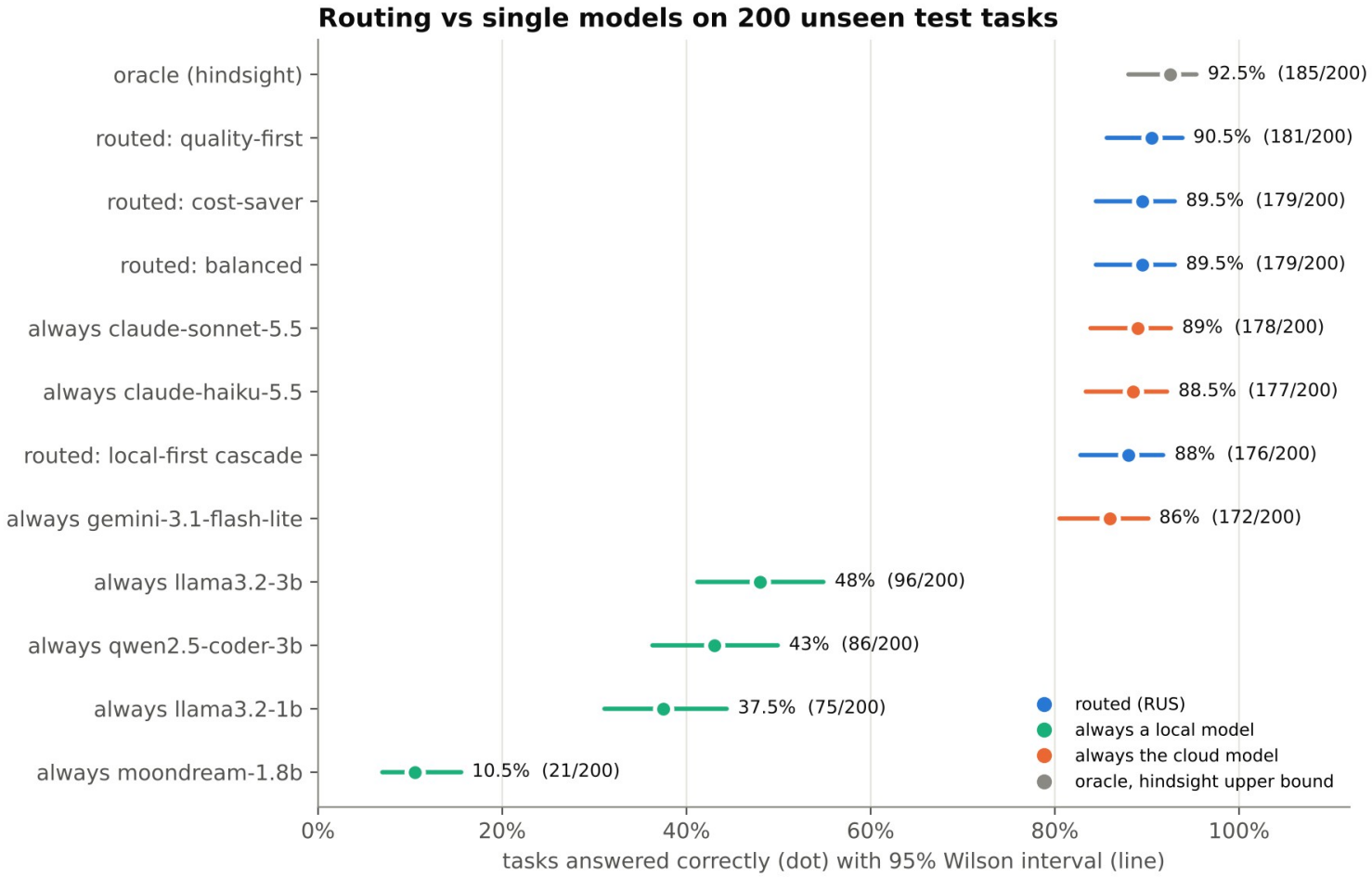
always Claude Sonnet 5.5, 178/200
McNemar p = 1.0: no difference

**−95%**

cost: $0.127 vs $2.506 per 1,000 questions
(₹11.1 vs ₹221)

Routing is significantly better than Gemini alone (86%, $p = 0.02$) and than the best local model (48%, $p < 0.001$). It is not more accurate than Sonnet; it is equally accurate for far less money.

# Every policy, with its uncertainty




Test tasks were never used to build the scorecards. Source: results/final/eval.json

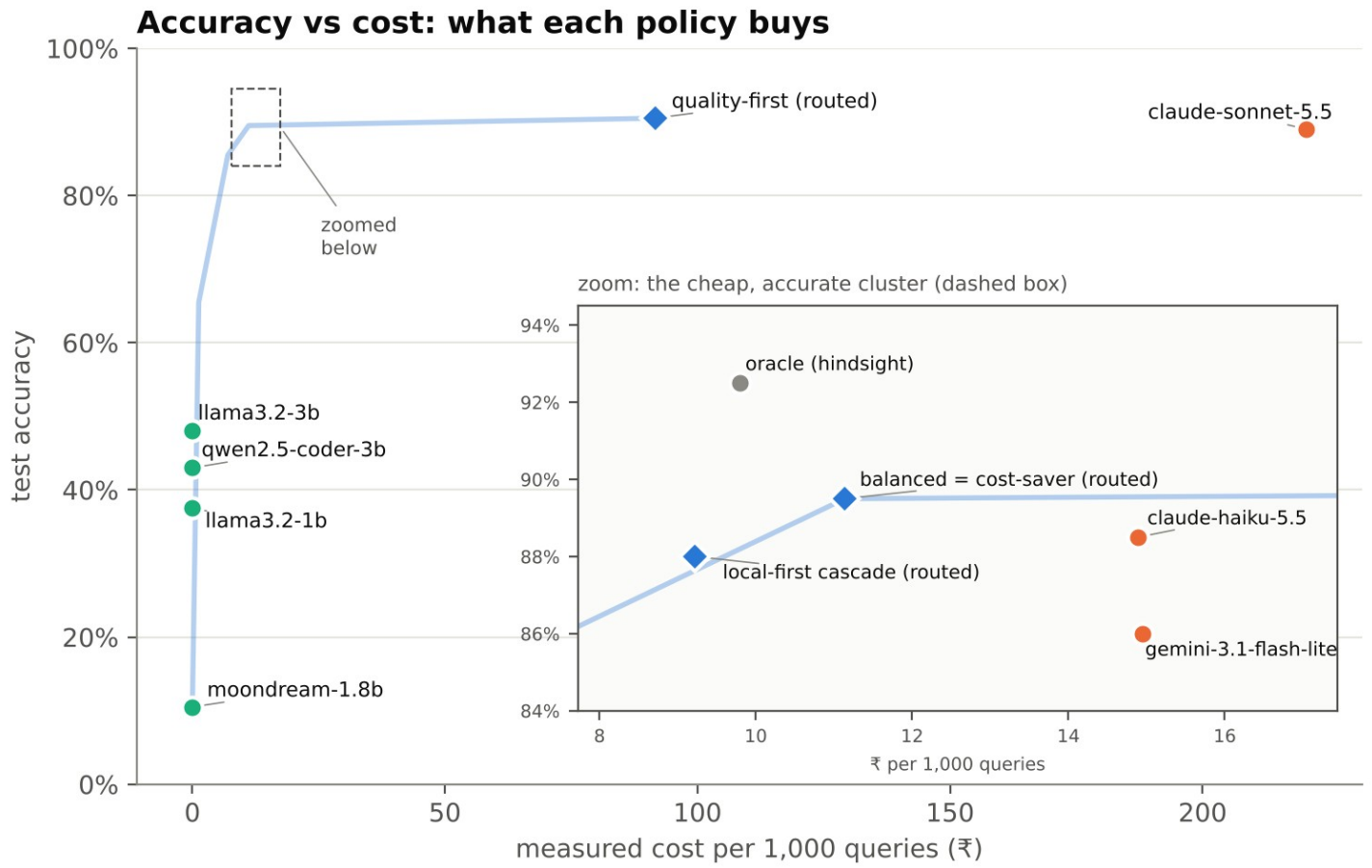
## How to read it

Dot = share of the 200 test tasks answered correctly; line = 95% Wilson interval.

Routed policies and the two Claude models overlap: statistically tied.

The oracle picks, with hindsight, the cheapest model that got each task right: 92.5%. It is the ceiling for any router on these models.

# What each policy buys, in rupees




Diamonds: routed policies; dots: one model always. Faint line: the router as the cost weight rises 0 to 0.9.
Cost = provider token counts x list price; local models cost ₹0; USD→INR 88.0. Source: results/final/eval.json

| Policy | Accuracy | ₹ / 1k |
|---|---|---|
| **Routed, balanced** | 89.5% | 11.1 |
| **Local-first cascade** | 88% | 9.2 |
| Always Haiku | 88.5% | 14.9 |
| Always Gemini | 86% | 14.9 |
| Routed, quality-first | 90.5% | 91.6 |
| Always Sonnet | 89% | 220.5 |

Raising the cost weight to 0.8 sends 63% of questions to local models and accuracy falls to 65.5%.

# Local first: keep an answer on the laptop if it checks out

### Coding

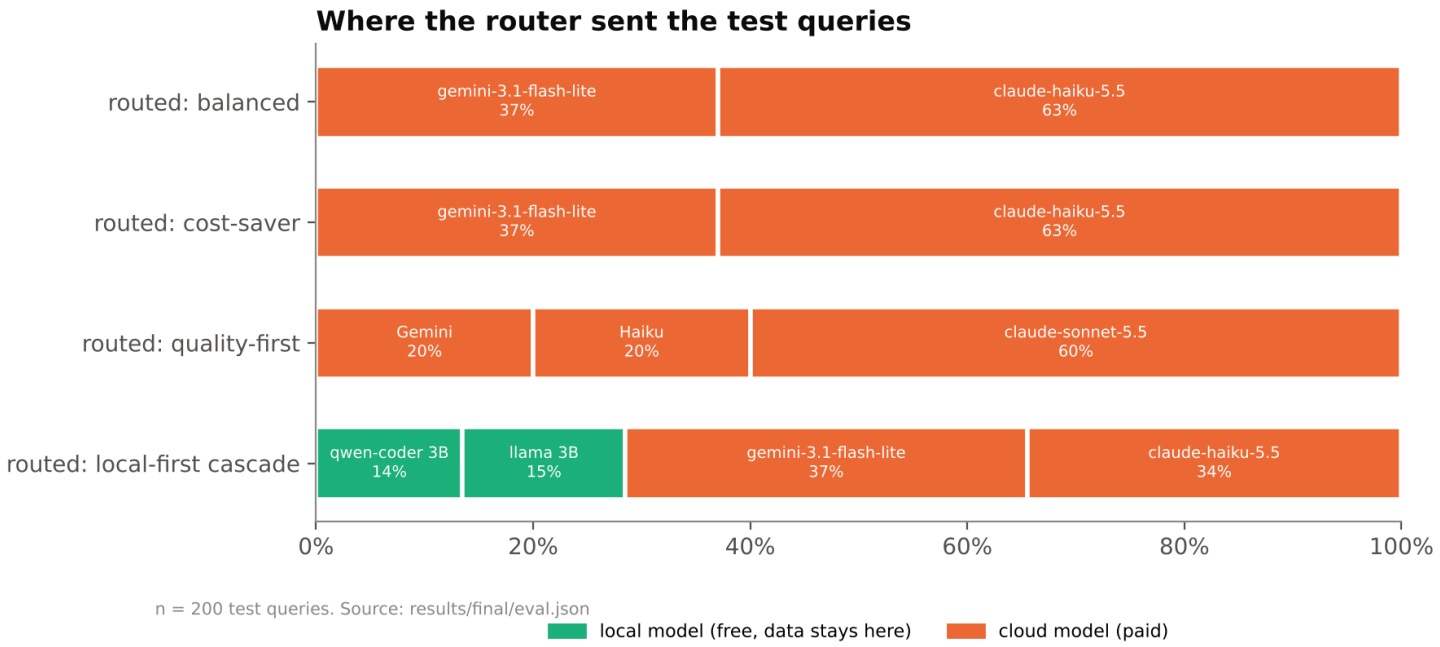
The local coder's code must pass the example assert in the user's own prompt.

### Maths

Two different local models must agree on the final number.

### Otherwise

Escalate to the model RUS picks. No check uses the right answer.




# 28.5%

of test questions answered on the laptop, at 88% accuracy (p = 0.79 vs Sonnet) and $0.105 per 1,000.

**Kept answers were right 54 times in 57.**

# “How is it 100%?” It was not, on fair prompts

| Task type | Before fix | After fix | 95% CI after |
|---|---|---|---|
| Coding | 37/40 | 40/40 | 91.2%–100% |
| Maths | 38/40 | 40/40 | 91.2%–100% |
| Factual search | 33/40 | 34/40 | 70.9%–92.9% |
| Writing | 18/40 | 40/40 | 91.2%–100% |
| Images | 40/40 | 40/40 | 91.2%–100% |
| **All** | 166/200 | 194/200 | 93.6%–98.6% |

## Where the 100% came from

A random split of our own templated prompts: near-copies sat in both training and test.

## The fix

Train on our prompts plus the public calibration prompts; test on the 200 public test prompts it never saw. Misrouted questions: 34 → 6.

Direct phrasings: 50/50, shown as “at least 93%” (95% CI), never as a bare 100%.

# Checking our own grading and stability

## Grader audit

A manual review found correct answers marked wrong (“Rachel Bilson” vs the accepted “Rachel Sarah Bilson”). We added a word-overlap rule with guards and report both graders.

**87.5%**

routed, strict grader

**87.5%**

Sonnet, strict grader

Same conclusion under either grader.

## Same tasks, run twice

| Model | Run 1 | Run 2 | Same mark | p |
|---|---|---|---|---|
| Claude Haiku 5.5 | 177/200 | 173/200 | 96% | 0.29 |
| Claude Sonnet 5.5 | 178/200 | 178/200 | 99% | 1.00 |
| Gemini 3.1 Flash-Lite | 170/198 | 170/198 | 100% | 1.00 |

Wording changes between runs; the pass or fail mark does not. Total paid API spend: about $1.87 under a $5 cap enforced in code.

# Ablation: what memory and retrieval each add

| Build | Switches | Memory recall | Document Q&A | Made-up answers | Points to a source |
|---|---|---|---|---|---|
| Baseline | none | 0/30 | 0/36 | 0/36 | 0/36 |
| Model A | routing | 0/30 | 0/36 | 5/36 | 0/36 |
| Model B | memory | 30/30 | 0/36 | 0/36 | 0/36 |
| Model C | RAG | 0/30 | 31/36 | 1/36 | 34/36 |
| **Model D** | routing + memory + RAG | 28/30 | 32/36 | 1/36 | 35/36 |

Each switch does its one job; the full build D keeps both: recall 28/30 (95% CI 78.7%–98.2%), document answers 32/36. Model B's 30/30 is shown with its interval, 88.6%–100%, not as a bare 100%.

# Privacy: personal context never left the laptop

## The rule

If a question carries a recalled memory, a retrieved passage or earlier turns, only local models are eligible. Cloud models are not even scored.

## Kept in one local folder

Memory database · document index · scorecards · cost ledger · egress log (size and hash of each cloud request, never its text)

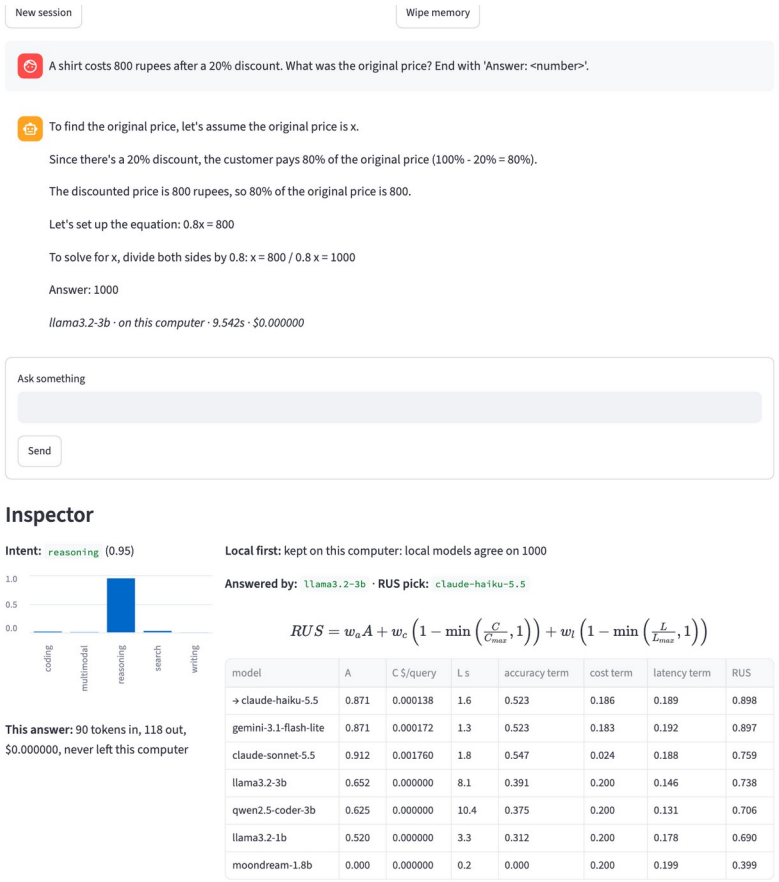
PRIVACY CHECK · BUILD D · ALL 8 REGISTERED MODELS AVAILABLE

# 0 of 54

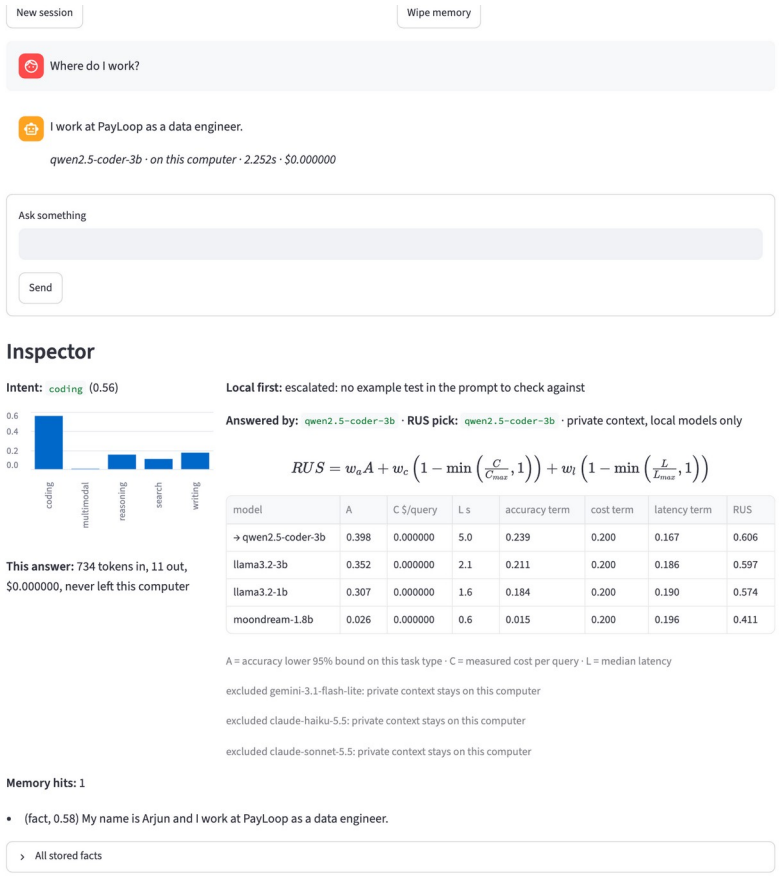
requests carrying personal context went to a cloud model.

Recall stayed at 28/30 with every cloud model available.

# Demo: every decision is shown




Maths: two local models agree on 1000, so the answer stays on the laptop. RUS alone would have picked Claude Haiku.




Memory: “Where do I work?” recalls a stored fact, so every cloud model is excluded and the answer never leaves the computer.

# Limitations and next steps

## What we do not claim

- 200 test tasks: intervals are about ±5 points
- On this benchmark, Haiku alone is nearly as good as routing
- The classifier still misroutes 6 of 200 public prompts
- Local models are 1B–3B; alone they reach 48%
- Image tasks are generated and easy for cloud models
- GPT-5 mini is registered but was not scored (no API credit)

## Next

- 7B–8B local models to raise the on-device share
- More cheap checks for the local-first step
- An embedding-based intent classifier
- Larger benchmark runs with the same scripts
- Recalibrate automatically when a provider changes price or model

# Thank you. Questions?

Same accuracy as the best cloud model, a fraction of the cost, and personal data that never leaves the laptop.

`github.com/teni-chintham/context-aware-ai-assistant`

Reproduce everything: run_final.command · MIT licence